Java Fundamentals

Java Backend Interview — Short Notes (Excl. Exception Handling, Multithreading, Collections)

1. Data Types & Variables

1.1 Primitive Data Types

Type	Size	Default	Range / Notes
byte	8 bit	0	-128 to 127
short	16 bit	0	-32,768 to 32,767
int	32 bit	0	-2^31 to 2^31-1. Most common integer type
long	64 bit	0L	-2^63 to 2^63-1. Use L suffix: 123L
float	32 bit	0.0f	Single precision IEEE 754. Use F suffix. Avoid for money
double	64 bit	0.0d	Double precision. Default decimal type. Still has floating-point errors
char	16 bit	'\u0000'	Unicode character. Single quotes. Range 0–65,535
boolean	JVM-dep	false	true or false only

✔ Primitives stored on Stack (local vars). Objects always on Heap

1.2 Wrapper Classes & Autoboxing

- Every primitive has a Wrapper: int→Integer, long→Long, double→Double, char→Character, boolean→Boolean
- **Autoboxing** — auto primitive→wrapper. **Unboxing** — auto wrapper→primitive
- **Integer cache** — Integer.valueOf(-128 to 127) returns cached instances. == can mislead outside range

```
Integer a=127; Integer b=127; a==b → TRUE (cached)
Integer x=128; Integer y=128; x==y → FALSE (new objects) — always use .equals()
```

- **NPE risk**: unboxing null wrapper throws NullPointerException

1.3 Type Casting

- **Widening (implicit)** — byte→short→int→long→float→double. No data loss, automatic
- **Narrowing (explicit)** — requires cast; possible data loss: (int)9.99 = 9 (truncates)
- **Object casting** — upcast implicit (Dog→Animal); downcast explicit (Animal→Dog). ClassCastException if wrong
- **Pattern matching instanceof (Java 16)**: if (a instanceof Dog d) { d.fetch(); }

1.4 var — Local Variable Type Inference (Java 10+)

- Compiler infers type from RHS. Static typing — NOT dynamic. Only for local variables

```
var list = new ArrayList(); // inferred as ArrayList
```

2. Operators

2.1 Operator Types

Category	Operators	Notes
Arithmetic	+ - * / %	/ on ints truncates. + also concatenates String
Relational	== != > < >= <=	== on objects compares references. Use .equals() for value
Logical	&& || !	Short-circuit: && stops if left false; || stops if left true

Bitwise	& | ^ ~ << >> >>>	<< left shift (x2). >> signed right shift. >>> unsigned right shift
Assignment	= += -= *= /= %= &= |= ^=	Compound: x+=5 same as x=x+5
Unary	+ - ++ -- !	Pre: ++i (increment then use). Post: i++ (use then increment)
Ternary	condition ? a : b	Concise if-else for expressions. Avoid nesting
instanceof	obj instanceof Type	Runtime type check. Pattern matching in Java 16+

2.2 Tricky Operator Behaviour

- **Integer division:** 7/2 = 3. Cast to get decimal: (double)7/2 = 3.5
- **String + number:** "Val: "+3+4 = "Val: 34". "Val: "+(3+4) = "Val: 7"
- **Pre vs Post:** int i=5; a=i++ → a=5,i=6. b=++i → b=7,i=7
- **Short-circuit:** null!=null && null.length()>0 → safe (right never evaluated)
- **& vs &&:** & evaluates both sides always; && short-circuits

3. Control Flow

3.1 if-else & switch

- **switch (classic)** — byte,short,int,char,String(Java7+),enum. Falls through without break
- **switch expression (Java 14+)** — returns value, arrow syntax, no fall-through, exhaustive

```
String result = switch(day) {
case MONDAY, TUESDAY -> "Weekday";
case SATURDAY, SUNDAY -> "Weekend";
default -> "Unknown"; };
```

3.2 Loops

Loop	Use When	Key Note
for	Known iterations	Init;condition;update — all optional. Infinite: for(;;)
while	Condition checked before each iteration	May never execute if condition starts false
do-while	Must execute at least once	Condition checked AFTER body
for-each	Iterate Iterable/array	No index. Cannot modify elements. Read-friendly

- **break** — exits innermost loop. **continue** — skips to next iteration
- **Labelled break** — breaks out of outer loop in nested loops

4. Strings

4.1 String Fundamentals

- String is a **class** (not primitive). Immutable — any modification creates new object
- **String Pool** — JVM maintains pool of string literals. Same literal reuses existing object

```
String a="hello"; String b="hello"; a==b → true (same pool ref)
String c=new String("hello"); a==c → false (different object)
a.equals(c) → true. c.intern()==a → true (forces into pool)
```

4.2 String vs StringBuilder vs StringBuffer

	String	StringBuilder	StringBuffer

Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes (immutable)	No	Yes (synchronized)
Performance	Slow concat in loop	Fastest	Slower (sync overhead)
Use when	Fixed / few concat	Single-thread building	Multi-thread building

• **+ in loop = O(n²)** — creates new String every iteration. Use StringBuilder.append() = O(n)

4.3 Key String Methods

Method	Description	Example
length()	Number of characters	"hello".length() → 5
charAt(i)	Character at index	"hello".charAt(1) → 'e'
substring(s,e)	From s (incl) to e (excl)	"hello".substring(1,3) → "el"
indexOf(str)	First index; -1 if absent	"hello".indexOf("ll") → 2
contains(str)	true if contains sequence	"hello".contains("ell") → true
startsWith/endsWith	Prefix/suffix check	"hello".startsWith("he") → true
toUpperCase/toLowerCase	Case conversion — returns new String	Original unchanged
trim() / strip()	Remove whitespace. strip() Unicode-aware (Java11+)	" hi ".strip() → "hi"
replace(old,new)	Replace all occurrences	"aabb".replace("bb","XX") → "aaXX"
split(regex)	Split into array by delimiter	"a,b,c".split(",") → [a,b,c]
equals/equalsIgnoreCase	Value comparison — never use == for Strings	Always prefer over ==
isEmpty/isBlank	isEmpty: len==0. isBlank: empty or whitespace (Java11+)	" ".isBlank()→true
String.format()	Format string	String.format("Hi %s", name)
join()	Join with delimiter (Java 8+)	String.join(", ","a","b")→"a, b"
repeat(n)	Repeat n times (Java 11+)	"ab".repeat(3) → "ababab"

5. Arrays

5.1 Fundamentals

• Fixed-size, zero-indexed, same type elements. Stored on Heap even for primitives

• **Default values:** int/long→0, double→0.0, boolean→false, Object→null

```
int[] arr = new int[5]; // [0,0,0,0,0]
int[] arr = {1,2,3,4,5}; // literal
arr.length // field not method. ArrayIndexOutOfBoundsException if idx < 0 or >= length
```

5.2 Arrays Utility (java.util.Arrays)

Method	Description
Arrays.sort(arr)	Sort ascending. O(n log n). Dual-Pivot Quicksort (primitives), TimSort (objects)
Arrays.binarySearch	Binary search on sorted array. Returns index or negative insertion point

Arrays.equals(a,b)	Element-by-element comparison (not ==, which compares references)
Arrays.fill(arr,val)	Fill entire array with a value
Arrays.copyOf(arr,n)	Copy with new length — truncates or pads with defaults
Arrays.toString(arr)	Readable output: [1,2,3]. For 2D: Arrays.deepToString()
Arrays.stream(arr)	Convert to Stream for functional operations

6. Enums

- Fixed set of named constants — type-safe, can have fields, constructors, methods
- All enums implicitly extend **java.lang.Enum**. Cannot extend other classes; can implement interfaces
- **Singleton guarantee** — enum is the safest Singleton in Java (JVM guaranteed)

Method/Feature	Description	Example
values()	Array of all constants	Status.values() → [PENDING,ACTIVE]
valueOf(String)	Constant by name — IllegalArgumentException if invalid	Status.valueOf("ACTIVE")
name()	Constant name as String — stable	Status.ACTIVE.name() → "ACTIVE"
ordinal()	0-based position — fragile (changes if order changes)	Avoid for logic — use custom fields
switch with enum	Works natively in switch/switch expression	switch(status){ case ACTIVE:... }
EnumSet/EnumMap	Highly efficient Set/Map for enum keys	EnumSet.of(DAY.MON, DAY.TUE)

```
enum Planet { MERCURY(3.303e+23, 2.4e6), EARTH(5.976e+24, 6.37e6);
private final double mass, radius;
Planet(double m, double r){ mass=m; radius=r; } }
```

7. Generics

- Type-safe compile-time parameterisation. **Type erasure** — generic info removed at runtime
- Cannot use primitives as type params: List<int> INVALID → use List<Integer>
- **Conventions:** T-Type, E-Element, K-Key, V-Value, R-Return

Wildcard	Meaning	Use When (PECS)
? extends T	T or any subtype — upper bound	Producer (reading): void print(List list)
? super T	T or any supertype — lower bound	Consumer (writing): void add(List list)
?	Unknown type	Read-only, no type constraint needed

```
class Box { private T value; public T get(){return value;} }
public > T max(T a,T b){ return a.compareTo(b)>=0?a:b; }
```

- **Type erasure:** List<String> and List<Integer> are both just List at runtime. Cannot do instanceof List<String>

8. Java Memory Model & Garbage Collection

8.1 JVM Memory Areas

Area	Stores	Notes

Stack	Local variables, method frames, primitives	Thread-private. LIFO. Fast. StackOverflowError on deep recursion
Heap	All objects and arrays (new keyword)	Shared. GC managed. Divided into Young/Old generation
Method Area	Class metadata, static variables, constant pool	Shared. Part of Heap in HotSpot
Metaspace	Class definitions (replaced PermGen in Java 8+)	Native memory, grows dynamically. OutOfMemoryError if OS runs out
PC Register	Current instruction address per thread	Thread-private
Native Stack	Native (C/C++) method calls via JNI	Thread-private

8.2 Garbage Collection

- JVM auto-reclaims memory of unreachable objects — no manual free()
- **Young Gen** — Eden + Survivor (S0,S1). New objects created here. Minor GC is fast and frequent
- **Old Gen (Tenured)** — long-lived objects promoted here. Major GC is slow, stop-the-world
- **GC algorithms:** G1GC (default Java 9+), ZGC (low-latency Java 15+), Shenandoah, ParallelGC
- **System.gc()** — hint only; NOT guaranteed. Never rely on it

```
void method(){
int x=5; // x on Stack
Person p=new Person(); // p (ref) on Stack; Person object on Heap
} // p popped from Stack → Person object eligible for GC
```

9. Java 8+ Key Features

9.1 Lambda Expressions

- Anonymous function implementing a functional interface. Syntax: **(params) -> expression**

```
Runnable r = () -> System.out.println("Hi");
Comparator c = (a,b) -> a.compareTo(b);
```

9.2 Functional Interfaces

Interface	Method	Description	Example
Predicate	test(T)→bool	Check condition	x -> x > 0
Function	apply(T)→R	Transform T to R	s -> s.length()
Consumer	accept(T)→void	Side effect / consume	x -> print(x)
Supplier	get()→T	Produce a value	() -> new Random()
BiFunction	apply(T,U)→R	Two inputs, one output	(a,b) -> a+b
UnaryOperator	apply(T)→T	Same input and output type	s -> s.toUpperCase()
BinaryOperator	apply(T,T)→T	Two same-type inputs	(a,b) -> a+b

9.3 Method References

Type	Syntax	Lambda Equivalent
Static method	Class::staticMethod	x -> Class.staticMethod(x)
Instance (bound)	obj::method	() -> obj.method()

Instance (unbound)	Class::instanceMethod	x -> x.method()
Constructor	Class::new	() -> new Class()

9.4 Stream API

- Functional pipeline. **Lazy** — intermediate ops don't execute until terminal op called
- **Intermediate (lazy, return Stream):** filter, map, flatMap, distinct, sorted, limit, skip, peek
- **Terminal (trigger execution):** collect, forEach, count, reduce, findFirst, anyMatch, allMatch, min, max

Operation	Description	Example
filter(pred)	Keep elements matching predicate	.filter(x -> x > 5)
map(fn)	Transform each element 1-to-1	.map(String::toUpperCase)
flatMap(fn)	Transform to stream then flatten 1-to-many	.flatMap(list -> list.stream())
distinct()	Remove duplicates (uses equals/hashCode)	.distinct()
sorted()	Sort naturally or with Comparator	.sorted(Comparator.reverseOrder())
collect(col)	Collect into List/Set/Map	.collect(Collectors.toList())
reduce(id,fn)	Aggregate to single value	.reduce(0, Integer::sum)
groupingBy	Group into Map by classifier	Collectors.groupingBy(Person::getDept)
anyMatch(pred)	True if any element matches	.anyMatch(x -> x > 10)
findFirst()	First element as Optional	.findFirst().orElse(null)

9.5 Optional

- Container for a possibly-absent value. Use as **return type only** — not field or parameter

Method	Description
Optional.of(v)	Non-null value. NPE if null passed
Optional.ofNullable(v)	Empty if null, present otherwise
orElse(default)	Return value or default — default always evaluated
orElseGet(supplier)	Lazy default — evaluated only when absent. Prefer over orElse for expensive defaults
orElseThrow()	Return value or throw NoSuchElementException (Java 10+)
ifPresent(consumer)	Execute consumer only if value present
map(fn)	Transform value if present, return new Optional
filter(pred)	Return empty Optional if predicate fails

9.6 New Features — Java 9 to 17

Version	Feature	Description
Java 9	Modules (JPMS)	module-info.java declares requires/exports. Modular JDK
Java 9	Private interface methods	Share code between default methods inside interface
Java 10	var	Local variable type inference. Static typing. Only for local vars
Java 11	String methods	isBlank(), strip(), lines(), repeat() added to String
Java 11	HTTP Client	java.net.http.HttpClient — async, HTTP/2. Replaces HttpURLConnection

Java 14	Switch expressions	Returns value, arrow syntax, no fall-through (standard Java 14)
Java 15	Text blocks	Multi-line """ strings — no escaping needed. Great for JSON/SQL/HTML
Java 16	Records	record Point(int x, int y){} — auto constructor, getters, equals, hashCode, toString
Java 16	Pattern instanceof	if(obj instanceof String s) — binds in one step
Java 17	Sealed classes	sealed class Shape permits Circle,Square — restricts hierarchy

10. Annotations

• Metadata for compiler, tools, or runtime frameworks. Do not affect execution directly

Annotation	Purpose
@Override	Compiler verifies method overrides parent. Compile error if it doesn't — prevents typos
@Deprecated	Marks element as outdated. Compiler warns callers
@SuppressWarnings	Suppresses compiler warnings: @SuppressWarnings("unchecked")
@FunctionalInterface	Ensures exactly one abstract method. Compile error if violated
@Retention	SOURCE (compile only), CLASS (bytecode), RUNTIME (reflection available)
@Target	Where annotation applies: TYPE, METHOD, FIELD, PARAMETER, etc.

11. Inner Classes

Type	Key Characteristic	Use Case
Inner (non-static)	Holds ref to outer instance. Accesses all outer members	Iterator, Builder with outer state
Static Nested	No outer instance reference. Instantiated independently	Helper/utility logically grouped with outer
Local Class	Scoped to block. Accesses final/effectively-final locals	Complex one-off logic inside method
Anonymous Class	Unnamed, inline, one-time. Cannot have constructor	Event listeners (pre-Java 8 lambdas)

12. Packages, Imports & Classpath

• **Package** — namespace mapping to directory structure. Convention: com.company.project.module

• **import** — compile-time only, no runtime cost. import static — imports static members

• **java.lang.*** — always auto-imported (String, System, Object, Math, etc.)

• **Classpath** — tells JVM where to find .class files and JARs

• **JAR** — ZIP of .class files + META-INF/MANIFEST.MF

• **Module (Java 9+)** — module-info.java declares requires (dependencies) and exports (public API)

13. JDK, JRE, JIT & Class Loading

13.1 JDK vs JRE vs JVM

Component	Full Name	Contains	Who Needs It

JVM	Java Virtual Machine	Class loader, bytecode interpreter, JIT compiler, GC, runtime memory areas	Everyone — runs Java programs
JRE	Java Runtime Environment	JVM + core libraries (java.lang, java.util, etc.) + supporting files	End users running Java apps
JDK	Java Development Kit	JRE + compiler (javac) + debugger (jdb) + tools (jar, javadoc, jconsole, jstack)	Developers writing/compiling Java

✔ JDK ⊃ JRE ⊃ JVM. From Java 9+, JRE is no longer distributed separately — JDK is the only download

13.2 How Java Code Runs — Compilation to Execution

- **Step 1 — Write:** Developer writes .java source file
- **Step 2 — Compile (javac):** Java compiler converts .java → .class (bytecode — platform-independent)
- **Step 3 — Class Loading:** JVM's ClassLoader loads .class file into memory
- **Step 4 — Bytecode Verification:** Verifier checks bytecode is safe and well-formed
- **Step 5 — Interpretation:** Interpreter executes bytecode line by line (slow initially)
- **Step 6 — JIT Compilation:** HotSpot detects frequently executed code (hot spots) → compiles to native machine code → runs at near-native speed

```
Source.java →[javac]→ Source.class (bytecode) →[JVM]→ Native machine code
```

13.3 JIT Compiler — Just-In-Time

- Part of JVM (HotSpot). Compiles hot bytecode to native machine code at **runtime**
- **Interpretation first** — JVM interprets bytecode initially. JIT kicks in after code runs enough times
- **Profiling** — JVM profiles code during interpretation, identifying hot spots (frequently executed loops/methods)
- **Tiered Compilation (Java 7+)** — uses C1 (fast, less optimised) then C2 (slow, highly optimised) compilers
- **Optimisations JIT performs:**
  - **Inlining** — replaces method call with method body to eliminate call overhead
  - **Dead code elimination** — removes code that can never execute
  - **Escape analysis** — if object doesn't escape method, allocate on Stack instead of Heap
  - **Loop unrolling** — reduces loop overhead by executing multiple iterations per cycle
  - **Constant folding** — evaluates constant expressions at compile time: 2*3 → 6
- **AOT (Ahead-Of-Time) compilation** — GraalVM native-image compiles to native binary at build time. Faster startup, lower memory. Used in Spring Native / Quarkus

✔ JIT makes Java competitive with C++ for long-running server applications after warmup. Startup slowness = interpretation phase before JIT kicks in

13.4 Class Loading — How It Works

- JVM loads classes **lazily** — a class is loaded only when first referenced
- **Three phases of class loading:**
  - **Loading** — read .class bytecode from source (filesystem, JAR, network) into Method Area
  - **Linking** — three sub-steps: Verification (bytecode safe?), Preparation (allocate static fields with defaults), Resolution (resolve symbolic references to actual memory addresses)
  - **Initialisation** — execute static initialiser blocks and assign static field values

13.5 ClassLoader Types & Hierarchy

ClassLoader	Loads	Location
Bootstrap ClassLoader	Core Java classes: java.lang.*, java.util.*, etc.	JDK's lib/rt.jar or modules (Java 9+). Written in native C — has no Java parent

Extension / Platform CL	Java extension classes and platform modules	lib/ext/ directory or --module-path (Java 9+)
Application (System) CL	Application classes and third-party JARs	Classpath (-cp) or module path. Default ClassLoader for your code
Custom ClassLoader	User-defined loading logic	Extend ClassLoader. Used by frameworks (Spring, Tomcat) to enable hot-reload, isolation

13.6 Delegation Model — Parent-First

• ClassLoaders follow **Parent Delegation Model** — always ask parent first, load yourself only if parent can't

■ Application CL → asks Extension CL → asks Bootstrap CL → if not found, loads itself

• **Why?** Prevents user code from replacing core Java classes (e.g., nobody can create their own java.lang.String)

• **Breaking delegation** — OSGi, Tomcat, Spring DevTools deliberately break parent delegation for class isolation and hot-reload

```
// ClassLoader delegation — pseudocode
Class loadClass(String name) {
Class c = findLoadedClass(name); // already loaded?
if (c == null) c = parent.loadClass(name); // ask parent first
if (c == null) c = findClass(name); // load myself
return c;
}
```

13.7 ClassLoader in Practice

• **Class.forName("com.example.Foo")** — loads and initialises class by name. Used in JDBC driver loading

• **Thread.currentThread().getContextClassLoader()** — get current thread's ClassLoader. Used by frameworks

• **ClassNotFoundException** — class not found on classpath. Check JAR dependencies

• **NoClassDefFoundError** — class was present at compile time but missing at runtime. Different from ClassNotFoundException

• **ClassCastException from different ClassLoaders** — same class loaded by two different ClassLoaders are different types. Common in app servers (Tomcat)

• **Hot reload (Spring DevTools / JRebel)** — custom ClassLoader reloads changed classes without restart

✔ ClassLoader isolation is why Tomcat can deploy multiple apps with conflicting library versions — each WAR gets its own ClassLoader

14. Frequently Asked & Tricky Interview Questions

Data Types & Variables

Q1. What is the difference between int and Integer?

→ int is primitive — stack, cannot be null, no methods. Integer is wrapper class — heap, can be null, has utility methods. Autoboxing converts between them

Q2. What is the Integer cache pitfall?

→ JVM caches Integer -128 to 127. == returns true in this range (same object). Outside range, new objects — == returns false. Always use .equals() for Integer comparison

Q3. Default value of instance vs local variables?

→ Instance: int/long→0, double→0.0, boolean→false, Object→null. Local variables: NO default — must initialise before use or compiler error

Q4. What is var? Is it dynamic typing?

→ Local variable type inference (Java 10). Compiler infers type at compile time — still static. Only for local variables, not fields, params, return types

Strings

Q5. Why is String immutable?

→ Security, thread safety, String pool reuse, hashCode caching (Strings used as HashMap keys)

Q6. String s1="hello"; String s2=new String("hello"); s1==s2?

→ false. s1 in pool, s2 is new heap object. Different references. s1.equals(s2) is true — same value

Q7. StringBuilder vs StringBuffer?

→ StringBuilder: single-threaded, faster. StringBuffer: synchronized, thread-safe, slower. Use StringBuilder in virtually all cases in modern Java

Q8. Why is + concatenation in a loop bad?

→ String is immutable — each + creates new object. N iterations = N intermediate objects. O(n^2). Use StringBuilder.append() — O(n)

Operators & Control Flow

Q9. What is short-circuit evaluation?

→ && stops if left is false. || stops if left is true. Right side not evaluated. Critical for null safety: obj!=null && obj.method()

Q10. Output: System.out.println("a"+1+2)?

→ "a12" — left to right. "a"+1="a1" then "a1"+2="a12". Use (1+2) to get "a3"

Q11. Switch expression vs switch statement?

→ Expression: returns value, arrow syntax, no fall-through, exhaustive. Statement: executes blocks, needs break, allows fall-through

Arrays & Generics

Q12. Array vs ArrayList?

→ Array: fixed size, can hold primitives, .length field. ArrayList: dynamic, objects only, rich methods. Array faster; ArrayList more flexible

Q13. What is type erasure?

→ Generic type params removed at compile time. List and List are both just List at runtime. Cannot do instanceof List

Q14. PECS principle?

→ Producer Extends Consumer Super. Reading from collection: ? extends T. Writing to collection: ? super T

Memory & JVM

Q15. Stack vs Heap?

→ Stack: thread-private, local vars and frames, LIFO, fast, StackOverflowError. Heap: shared, all objects (new), GC managed, OutOfMemoryError

Q16. What causes StackOverflowError?

→ Deep/infinite recursion fills Stack — each call adds a frame. Fix: add base case, convert to iterative

Q17. Minor GC vs Major GC?

→ Minor: Young Gen, fast, frequent. Major/Full: Old Gen, slow, longer stop-the-world pause

Q18. What is a memory leak in Java?

→ Objects no longer needed but still referenced — GC cannot collect. Causes: static collections, unclosed resources, listeners not removed, ThreadLocal not cleared

Java 8+ Features

Q19. map() vs flatMap()?

→ map(): 1-to-1 transform. flatMap(): 1-to-many — maps to stream then flattens. Use flatMap for nested collections/Optional chains

Q20. orElse() vs orElseGet()?

→ orElse(default): default always evaluated even if value present. orElseGet(supplier): lazy — evaluated only when value absent. Use orElseGet for expensive defaults

Q21. Intermediate vs terminal Stream operations?

→ Intermediate: lazy, return Stream, don't execute (filter/map/sorted). Terminal: trigger execution, return result (collect/forEach/count). Without terminal op, nothing runs

Q22. What is a functional interface?

→ Interface with exactly one abstract method. @FunctionalInterface annotation enforces this. Lambda target. e.g., Runnable, Comparator, Predicate

JDK / JRE / JIT

Q23. Difference between JDK, JRE, and JVM?

→ JVM: runs bytecode (interpreter + JIT + GC + memory). JRE: JVM + core libraries (needed to run). JDK: JRE + compiler + dev tools (needed to develop). JDK ⊃ JRE ⊃ JVM

Q24. What does the Java compiler (javac) produce?

→ Bytecode — platform-independent .class files. Not native machine code. JVM interprets/JIT-compiles bytecode to native code at runtime

Q25. What is JIT compilation? Why is Java slower on startup?

→ JIT (Just-In-Time) compiles hot bytecode to native code at runtime. Startup is slower because JVM starts interpreting — JIT kicks in after profiling identifies hot spots

Q26. What is tiered compilation?

→ Java 7+ uses two JIT compilers: C1 (client, fast compile, less optimised) for quick startup, C2 (server, slow compile, highly optimised) for hot code. JVM moves code from C1 to C2 automatically

Q27. What optimisations does JIT perform?

→ Inlining (eliminate call overhead), dead code elimination, escape analysis (stack vs heap allocation), loop unrolling, constant folding

Q28. What is AOT compilation (GraalVM)?

→ Compiles Java to native binary at build time (ahead-of-time). Near-instant startup, lower memory. Used in Spring Native, Quarkus. Trade-off: slower build, less JIT optimisation at runtime

Class Loading

Q29. What are the three phases of class loading?

→ Loading (read .class into memory), Linking (Verification + Preparation + Resolution), Initialisation (run static blocks, assign static fields)

Q30. What are the types of ClassLoaders?

→ Bootstrap (core Java, native C), Extension/Platform (ext libs, Java 9+ modules), Application/System (your code + classpath JARs), Custom (user-defined, used by frameworks)

Q31. What is the Parent Delegation Model?

→ ClassLoader always asks parent first before loading itself. Bootstrap → Extension → Application. Prevents user code replacing core Java classes (e.g., can't override java.lang.String)

Q32. When is a class initialised?

→ First: new instance created, static method called, static field accessed, class is top-level and main() runs, or reflection. Lazy — only when first actively used

Q33. Difference between ClassNotFoundException and NoClassDefFoundError?

→ ClassNotFoundException: class not found on classpath at runtime (checked exception, Class.forName()). NoClassDefFoundError: class was present at compile time but missing at runtime (JVM error)

Q34. How does Tomcat achieve class isolation between webapps?

→ Each WAR gets its own WebAppClassLoader. Breaking parent delegation — webapp loads its own libs first. Allows different apps to use conflicting versions of same library

Q35. What is Class.forName() used for?

→ Loads and initialises a class by name at runtime. Classic use: JDBC — Class.forName("com.mysql.Driver") registers driver without compile-time dependency

Q36. What is hot-reload and how does it work?

→ Spring DevTools / JRebel use custom ClassLoader. On file change, throw away old ClassLoader (and its classes), create new one and reload changed classes. Possible because ClassLoaders are garbage-collected

Enums, Annotations & Misc

Q37. Why use enum over int constants?

→ Type-safe, readable, switch-compatible, can have fields/methods, IDE autocompletion, immutable by design

Q38. Safest Singleton implementation?

→ Enum singleton: enum Singleton { INSTANCE; } — JVM guarantees one instance, thread-safe, serialisation-safe, reflection-safe

Q39. What is @Retention RUNTIME used for?

→ Makes annotation available via reflection at runtime. Needed for Spring (@Autowired, @Transactional), JPA (@Entity, @Column), JUnit (@Test) to inspect and act on annotations at runtime

Q40. Inner class vs static nested class?

→ Inner: non-static, holds implicit ref to outer instance, can access outer members, more memory. Static nested: no outer ref, instantiated independently, preferred when outer ref not needed

Advanced & Tricky

Q41. Is Java pass-by-value or pass-by-reference?

→ Always pass-by-value. Primitives: value copied. Objects: copy of reference copied — method can modify object state but cannot change caller's reference variable

Q42. What is the difference between final, finally, finalize()?

→ final: keyword for constant/non-overridable/non-extendable. finally: block that always runs in try-catch. finalize(): deprecated Object method called by GC before collection — avoid

Q43. What causes OutOfMemoryError?

→ Heap full (too many objects), Metaspace full (too many class definitions — class leak), or StackOverflow (but that's a different error). Common cause: memory leak via uncollectable references

Q44. What is escape analysis?

→ JIT optimisation that determines if an object 'escapes' a method (returned or stored elsewhere). If it doesn't escape, JIT allocates it on Stack instead of Heap — faster allocation, no GC needed

Q45. What is a text block (Java 15)?

→ Multi-line string literal with triple quotes """. Preserves formatting, no escape needed for quotes/newlines. Great for SQL, JSON, HTML embedded in code

Q46. What are sealed classes (Java 17)?

→ sealed class Shape permits Circle, Square — only listed classes can extend. Enables exhaustive switch pattern matching. Compiler knows all subtypes

Q47. What is the difference between compile-time and runtime errors?

→ Compile-time: syntax errors, type mismatches — caught by javac. Runtime: NPE, ArrayIndexOutOfBounds, ClassCastException — occur during JVM execution

Q48. What is Metaspace? How is it different from PermGen?

→ Metaspace (Java 8+) stores class metadata in native memory — grows dynamically. PermGen was fixed-size heap area — caused OutOfMemoryError on heavy class loading (e.g., JSP compilation)

Q49. How does the JVM ensure type safety at runtime?

→ Bytecode verifier checks code before execution. ClassLoader verifies class structure. Runtime checks for ClassCastException, ArrayStoreException. Generics add compile-time checks (erased at runtime)

Q50. Write-once run-anywhere — how does Java achieve this?

→ javac compiles to bytecode (not native code). Bytecode is platform-independent. JVM (platform-specific) translates bytecode to native instructions. Different JVMs for Windows/Linux/Mac — same .class files run everywhere